

MIT 103 · WEEK 2

Systems, Roles & Development Methodologies

How systems are classified, who the analyst is, and the three ways teams build software

College of Engineering and Computational Sciences

Master of Information Technology · Partido State University

Instructor: Rowell John B. Artiaga

Where We Left Off — and Where We're Going

Week 1 set the frame. Week 2 gives you the vocabulary every systems analyst works in.



LAST WEEK

Course orientation

Outcomes, grading, the 18-week map, and our running case: the PARSU Student Clinic Appointment System.



TODAY

Systems, roles & methods

The five classes of computer-based systems, how new technology reshapes user behaviour, the three analyst roles, and SDLC vs Agile vs OOSAD.



NEXT WEEK

Modeling organizations

Context diagrams, entity-relationship models, use cases — and your first individual System Proposal activity.

Learning Outcomes for Week 2

Syllabus-aligned — mapped to Program Outcome indicators a.1, a.3 and d.2



Recall the basic types of computer-based systems

TPS, OAS/KWS, MIS, DSS and ESS — and what each one is actually for.

PO a.1



Explain how users and new technologies change system dynamics

Cloud, mobile, AI and IoT do not just replace tools — they rewrite behaviour.

PO a.1



Distinguish the three roles of a systems analyst

Consultant, supporting expert, and agent of change.

PO d.2



Compare SDLC, Agile and object-oriented approaches

Know the phases, the strengths, the limits — and when each one fits.

PO a.3



Justify a methodology choice with defensible criteria

Argue from project characteristics, not from preference.

PO d.2



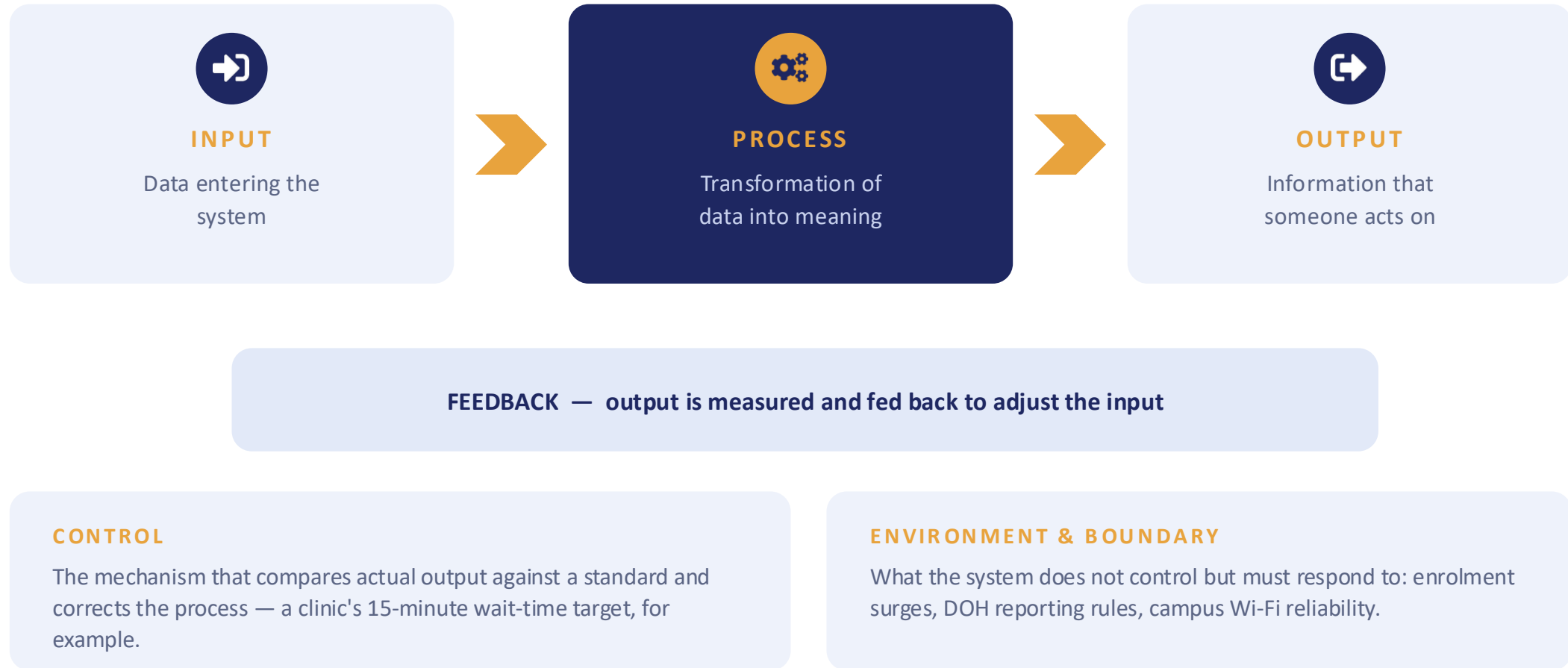
PART 1

What Exactly Is a System?

Before we classify systems, we need to agree on what one is.

The Anatomy of a System

Every information system — from a paper logbook to an AI scheduler — has the same five parts



Open Systems, Closed Systems, and Why It Matters

The boundary you draw determines the requirements you will be held to



RELATIVELY CLOSED

Controlled inputs, controlled access

- Only defined, pre-formatted inputs are accepted
- Access is restricted to authorised roles
- Outputs go to a known, fixed set of consumers
- Example: a payroll run, a bank's core ledger



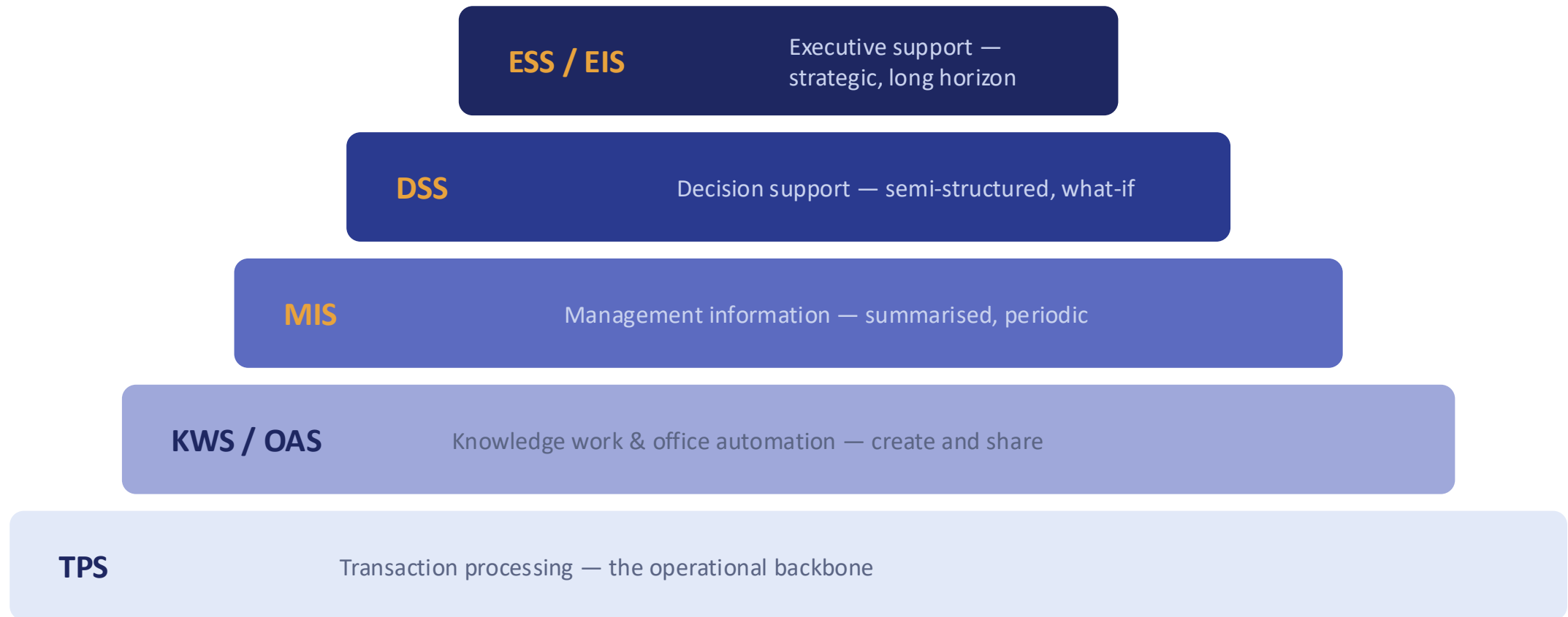
OPEN

Exposed to an unpredictable world

- Accepts varied, messy input from many sources
- Interacts continuously with its environment
- Must be designed for exceptions, not just the happy path
- Example: our clinic appointment system

The Five Classes of Computer-Based Systems

Organisations run a layered stack — each layer serves a different decision and a different user



▲ *Fewer users, broader scope, less structured decisions*

▼ *More users, narrower scope, highly structured tasks*

TPS — The Transaction Processing Backbone

High volume, highly structured, zero tolerance for loss. Everything else is built on this.

WHAT MAKES A TPS A TPS



High volume

Thousands of identical events a day



Highly structured

Every transaction has the same shape



Must not lose data

A dropped transaction is a real-world error



Operational users

Clerks, tellers, front-desk staff

IN THE WILD



PARSU Student Clinic

Each walk-in logged: student ID, complaint, time in, time seen. This is the raw material for every report the clinic will ever produce.



Bank ATM network

Withdrawal, deposit, balance enquiry — millions per day, each one must settle exactly once.



Retail point-of-sale

Every scan updates inventory, revenue and reorder triggers simultaneously.

MIS vs DSS — A Distinction Students Routinely Blur

Same data, different question. Know which one you are being asked to build.

MIS

Management Information System

ANSWERS

“What happened?”

DECISION TYPE

Structured, recurring

OUTPUT

Scheduled reports, dashboards, exception lists

USER

Middle management

CLINIC EXAMPLE

Monthly report: consultations by day, top five complaints, average wait time

DSS

Decision Support System

ANSWERS

“What should we do?”

DECISION TYPE

Semi-structured, one-off

OUTPUT

Models, scenarios, sensitivity analysis

USER

Analysts and decision makers

CLINIC EXAMPLE

If we add a second nurse on Mondays, what happens to peak-hour wait time?

The Rest of the Stack — and Where AI Now Sits

Knowledge work, office automation, executive support, and the intelligent layer that cuts across all of them



KWS

Knowledge Work Systems

Help professionals create new knowledge — CAD for engineers, statistical and modelling environments for researchers, EHR-linked clinical tools.



OAS

Office Automation Systems

Move and share existing knowledge — email, shared calendars, document management, video conferencing, workflow routing.



ESS

Executive Support Systems

Highly aggregated, visual, exception-driven. Built for scanning the environment, not for detail. Drill-down on demand.



ES / AI

Expert & Intelligent Systems

Capture and apply expertise — rule engines, and now ML models for triage, forecasting and anomaly detection embedded across every layer.

One Case, Five Layers

The PARSU Student Clinic Appointment System is not one system — it is a stack of five

TPS

Book, check in, record consultation, close visit

OAS / KWS

SMS + email reminders; clinician notes and referral templates

MIS

Weekly utilisation report to the Clinic Head and Student Affairs

DSS

Staffing simulator: what-if we shift a nurse to the Monday morning peak?

ESS

One dashboard tile for the VP: service coverage vs enrolment growth

Design question we will carry all semester: which layer do we build first, and what does that decision cost us later?



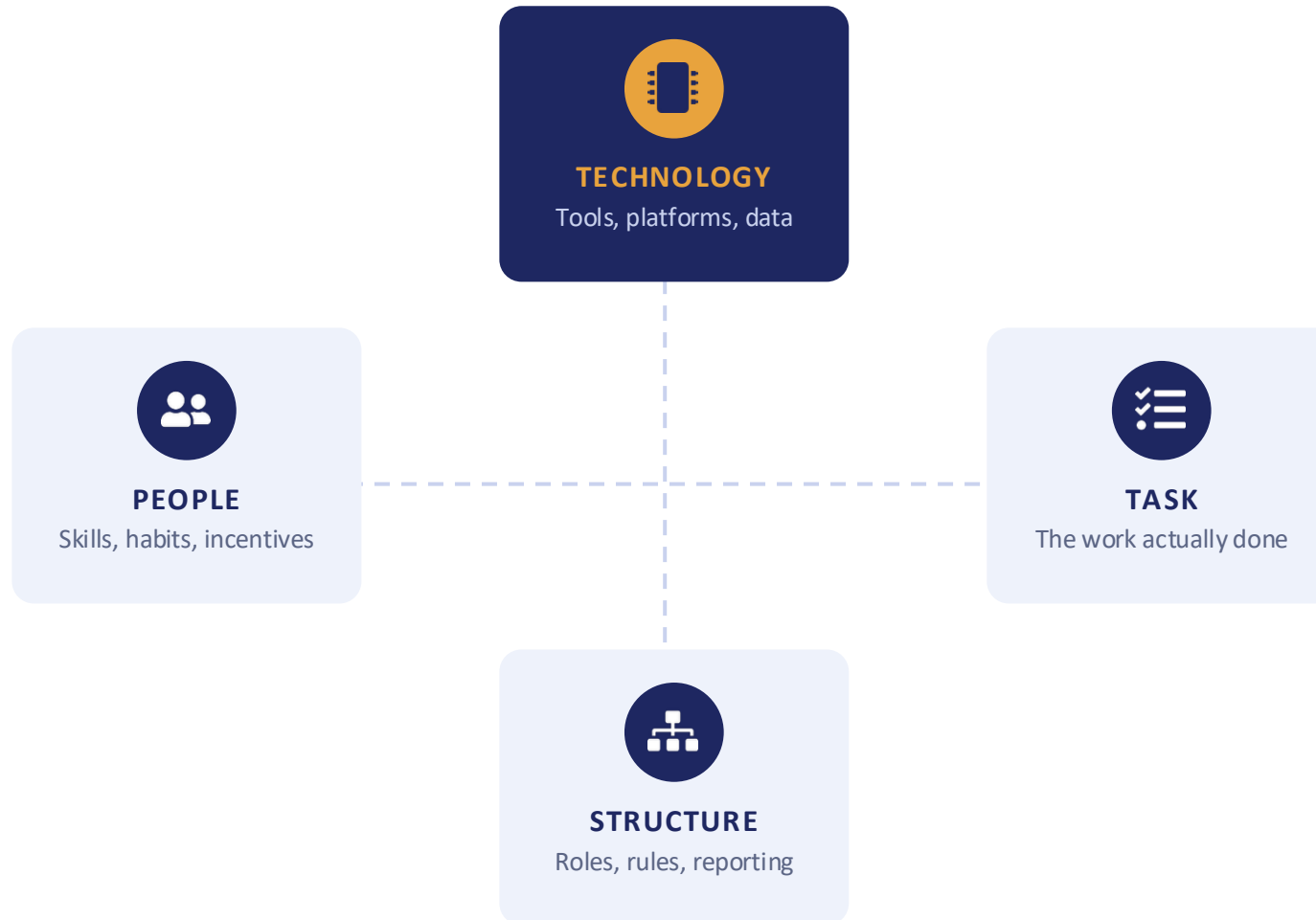
PART 2

Users, Technology, and System Dynamics

Install new technology and you do not get the old system running faster. You get a different system.

Change One Corner, You Change All Four

Leavitt's diamond — the reason technically perfect systems still fail on go-live day



WHAT THIS COSTS YOU

Give the clinic an online booking system and the queue does not disappear — it moves online, and no-shows rise.

The receptionist's job changes from gatekeeper to exception handler.

A new escalation rule is needed for students who book and never arrive.

None of that is in the software requirements. All of it is in your job.

Five Technologies Rewriting the Analyst's Job

Each one moves work across the system boundary — and creates a new class of requirement



Cloud

Capacity becomes a monthly cost, not a capital purchase. New requirements: data residency, vendor exit, cost governance.



Mobile

The user is no longer at a desk. New requirements: intermittent connectivity, small-screen data entry, device security.



AI / ML

Systems now make recommendations. New requirements: explainability, bias testing, human override, model drift monitoring.



IoT / Sensors

Input arrives without a human. New requirements: stream volume, sensor failure modes, physical security.



E-commerce & Platforms

External parties transact directly. New requirements: 24/7 availability, self-service recovery, trust and payment integration.

Notice the pattern: every one of these adds non-functional requirements — the kind discovered late and costliest to retrofit.

Same Clinic, New Technology, Different System

A concrete before-and-after — read it as a change in behaviour, not a change in tooling

DIMENSION	BEFORE — PAPER LOGBOOK	AFTER — APPOINTMENT SYSTEM
Getting seen	Queue physically from 7:00 a.m.	Book a slot from your phone the night before
Who holds the record	A bound logbook at the front desk	A shared record the student can also see
Receptionist's role	Controls the queue	Handles exceptions and no-shows
Failure mode	Students give up and leave	Students book and do not arrive
New work created	None	Reminder policy, cancellation rules, data-privacy consent

The last row is the one analysts miss. New technology does not only remove work — it manufactures new work.



PART 3

The Roles of a Systems Analyst

You will be asked to be three different people — sometimes in the same meeting.

Three Roles, One Job Title

Kendall & Kendall's classic framing — and the risk that comes with each role



THE OUTSIDER'S ADVANTAGE

Consultant

Brought in from outside the day-to-day. You see the process without the assumptions the staff no longer notice.

Risk: you never learn the organisational culture well enough to know which “inefficiency” is actually a safeguard.



THE TECHNICAL AUTHORITY

Supporting Expert

You supply the deep technical judgement on hardware, platforms, data and integration that the organisation cannot supply itself.

Risk: you slip into deciding for the client instead of equipping them to decide.



THE MOST DEMANDING ROLE

Agent of Change

Your presence changes the organisation. You plan the change, help people through it, and stay accountable after go-live.

Risk: you deliver working software into an organisation that was never prepared to use it.

What the Role Actually Demands

Technical skill gets you hired. The other three columns decide whether your system is used.



ANALYTICAL

- Systems thinking — see wholes, not parts
- Problem framing before problem solving
- Comfort with ambiguity and incomplete data
- Evidence-based reasoning



TECHNICAL

- Platforms, data and integration patterns
- Modelling notation (DFD, ERD, UML)
- Enough depth to be credible with developers
- Continuous re-learning



MANAGERIAL

- Scope, schedule and resource judgement
- Risk identification and escalation
- Documentation others can act on
- Vendor and stakeholder management



INTERPERSONAL & ETHICAL

- Listening far more than presenting
- Writing for non-technical readers
- Data privacy and informed consent
- Saying no with a defensible reason

Under the Data Privacy Act of 2012, the analyst who designs the collection is accountable for the collection. Ethics is a design activity, not a compliance sign-off.

The Analyst Sits in the Gap

Your value is not in either world — it is in the translation between them

THE ORGANISATION

- Clinic Head
- Front-desk staff
- Students (end users)
- Student Affairs
- Data Protection Officer



THE SYSTEMS ANALYST

- Elicits what people mean, not just what they say
- Converts intent into a model developers can build
- Converts constraints back into language the business can decide on
- Owns the trade-off conversation



THE BUILD TEAM

- Developers
- Database designers
- QA / testers
- Infrastructure
- Vendors



PART 4

Three Development Methodologies

SDLC, Agile, and object-oriented systems analysis and design — the core of today's session.

The SDLC — Seven Phases

The structured approach: each phase produces a deliverable that gates the next



Phases overlap in practice, and evaluation feeds back into phase 1 — the SDLC is a cycle, not a one-way waterfall.

DELIVERABLES YOU WILL PRODUCE IN THIS COURSE

Problem definition · Requirements document · DFDs and data dictionary · Design specification · Test plan · Implementation and evaluation report

SDLC — Where It Wins and Where It Hurts

A structured approach is a bet that requirements will hold still. Sometimes that bet is right.



STRENGTHS

- Clear phase gates and documented deliverables
- Easy to plan, budget and audit — fits procurement and regulated settings
- Strong traceability from requirement to test case
- Survives team turnover, because knowledge is written down
- Works well when the domain is well understood and stable



LIMITS

- Late feedback — users often see working software only near the end
- Expensive to change once a phase is signed off
- Assumes requirements can be fully known up front
- Documentation can become the goal rather than the system
- Poor fit for exploratory or rapidly shifting problems

Use it when: the domain is stable, compliance requires an audit trail, and the cost of a wrong build is higher than the cost of slow feedback.

The Agile Approach

Four values, four control variables — Agile does not remove constraints, it makes them explicit and negotiable

FOUR CORE VALUES



Communication

Talk continuously; the customer is on the team



Simplicity

Build the simplest thing that could work today



Feedback

Short cycles so mistakes surface in days, not months



Courage

Change the design, delete the code, tell the truth about scope

FOUR CONTROL VARIABLES



TIME



COST



QUALITY



SCOPE

The rule: management sets three, the team sets the fourth.

If the date, the budget and the quality bar are fixed, then scope is what moves. Pretending all four are fixed is how projects fail quietly.

In practice, quality is the variable teams sacrifice by accident. Agile makes you sacrifice scope on purpose instead.

Agile in Motion — Five Stages, Tight Loops

The whole method is a machine for shortening the distance between a decision and its consequences



FOUR AGILE ACTIVITIES, REPEATED EVERY ITERATION



Object-Oriented Systems Analysis and Design

Model the world as interacting objects, then let the model become the system

THE CORE IDEA

An object bundles data and the behaviour that acts on it.

A class is the template; an object is an instance of it.

Encapsulation hides internals; inheritance reuses them; polymorphism lets one message mean different things to different objects.

Because analysis, design and code share one vocabulary, the model does not have to be re-translated at every phase.

WHAT YOU ACTUALLY PRODUCE — UML



Use case diagram + scenarios

Who uses the system, and to achieve what



Class diagram

The objects, their attributes, and their relationships



Sequence diagram

How objects collaborate over time to satisfy a use case



State & activity diagrams

How an object changes state; how the workflow branches



CRC cards

Class, Responsibility, Collaborator — a low-tech workshop tool

OOSAD is incremental and iterative — you build one use case at a time and revisit the model constantly, which is why it pairs naturally with Agile delivery.

Head to Head

Read down a column to understand a method; read across a row to make a choice

	SDLC	AGILE	OOSAD
Core unit of work	Phase	Iteration / story	Use case & object
Requirements	Fixed early, signed off	Emergent, re-prioritised each cycle	Captured as use cases, refined iteratively
User involvement	Concentrated in early phases	Continuous — customer on the team	Recurring, at use-case boundaries
Primary artefacts	Requirements doc, DFDs, design spec	Backlog, working software, tests	UML models, CRC cards, class library
Handles change	Poorly and expensively	By design	Well, through encapsulation and reuse
Documentation	Heavy and formal	Light, just-enough	The model is the documentation
Best fit	Stable, regulated, large and contractual	Unclear or shifting requirements, co-located team	Complex domains with reusable components
Main risk	Builds the wrong thing thoroughly	Scope drift; weak audit trail	Steep modelling learning curve

How to Choose — Six Questions, Not a Preference

This is the reasoning your homework and your Week 3 proposal will be graded on



How well are the requirements understood?

Well understood → SDLC. Discovered through use → Agile.



How costly is late feedback?

If a wrong build is catastrophic and slow to fix, front-load with SDLC.



Is there a regulatory or contractual audit trail?

Procurement, DOH reporting or a signed SOW pushes you toward SDLC.



Can a real user sit with the team continuously?

No on-site customer means Agile loses its main feedback mechanism.



How complex and reusable is the domain model?

Many interacting entities with shared behaviour → OOSAD pays for itself.



What is the team's actual experience?

A method the team cannot execute is worse than a simpler one they can.

Hybrids are normal and legitimate. What is not legitimate is choosing without stating why.

Lab Session — The Methodology Bake-Off

60 minutes, in groups of three. Bring your reasoning, not your preference.

10 min

Draw your brief

Each group receives one of four project briefs: a DOH-reportable clinic records upgrade, a student marketplace startup, a bank's core ledger migration, and an AI triage assistant.

20 min

Score the decision matrix

Rate your brief against the six choice criteria on the worksheet. Weight them. Compute a score for SDLC, Agile and OOSAD.

15 min

Build the counter-argument

Argue the case for the method your matrix rejected. If you cannot, your matrix is probably biased.

15 min

Defend in 3 minutes

One recommendation, two supporting criteria, one named risk and its mitigation. The class challenges.

Deliverable at the end of the hour: one completed decision matrix per group, photographed and posted to Google Classroom.

Worked Example — Scoring the Clinic System

Rate each method 1–5 against the criterion, multiply by the weight, and total the column

Criterion	Weight	SDLC	Agile	OOSAD
Requirements clarity at start	0.20	4 = 0.80	3 = 0.60	3 = 0.60
Tolerance for late feedback	0.25	2 = 0.50	5 = 1.25	4 = 1.00
Audit / DOH reporting trail	0.15	5 = 0.75	2 = 0.30	3 = 0.45
On-site user availability	0.15	3 = 0.45	4 = 0.60	4 = 0.60
Domain complexity & reuse	0.10	3 = 0.30	3 = 0.30	5 = 0.50
Team experience with method	0.15	4 = 0.60	3 = 0.45	2 = 0.30
WEIGHTED TOTAL	1.00	3.40	3.50	3.45

The scores are almost identical — and that is the lesson. When a matrix produces a near-tie, the honest recommendation is a hybrid: SDLC-style requirements and sign-off for the DOH-reportable records module, Agile iterations for the student-facing booking flow, UML use cases as the shared modelling language across both. State the split and the reason.

Five Things to Carry Out of This Room

1

Systems come in layers.

TPS, OAS/KWS, MIS, DSS, ESS. Ask which layer you are being asked to build — and which one it depends on.

2

Technology changes behaviour, not just tooling.

Every corner of the diamond moves. Requirements you cannot see in the software spec will decide whether the system is used.

3

The analyst is three people.

Consultant, supporting expert, agent of change. Know which hat you are wearing in each conversation.

4

Methodologies are bets about change.

SDLC bets requirements will hold. Agile bets they will not. OOSAD bets the model is worth the investment.

5

Choosing without justifying is the real failure.

Any of the three can succeed. Selecting one without stated criteria is what gets projects into trouble.

Homework — The Methodology Decision Brief

Individual · due at the start of Week 3 · this becomes the first section of your semester System Proposal

THE TASK

Choose one real information system in an organisation you can actually access. Classify it, analyse who it changed, and recommend a development methodology for its next major release — with a weighted decision matrix and a named risk. Then defend it in five minutes.

- | | | | |
|----------|--------------------------|--------|--|
| A | System Profile | 1 page | Name the organisation and system. Classify it against the five layers. Draw its input–process–output–feedback–control diagram and mark the boundary. |
| B | Dynamics Analysis | 1 page | Apply Leavitt's diamond. What changed in task, people and structure when this technology arrived — or what will change? Name one behaviour the system created. |
| C | Role Reflection | ½ page | Which of the three analyst roles would this project most demand of you, and what is the specific risk you would be exposed to? |
| D | Decision Matrix | 1 page | Score SDLC, Agile and OOSAD against the six criteria with your own weights. Justify each weight in one sentence. |
| E | Recommendation | ½ page | One method or a stated hybrid, two supporting criteria, one named risk and its mitigation. Hybrids must specify the split. |

Submit as a single PDF to Google Classroom, plus a 5-slide deck for your Week 3 individual presentation. Worksheet and rubric are posted with these slides.

Pace Yourself — and Know How You Are Marked

A day-by-day plan on the left, the scoring breakdown on the right

SEVEN-DAY PLAN

Day 1

Choose the system. Confirm you can reach one person who uses it daily.

Day 2

Draft Part A — classify and diagram. Mark the boundary deliberately.

Day 3

Talk to that user for 20 minutes. Take notes on what changed for them.

Day 4

Write Part B and Part C from those notes.

Day 5

Build the matrix. Set weights first, score second — never the reverse.

Day 6

Write the recommendation. Argue the losing case once, then discard it.

Day 7

Cut the deck to five slides. Rehearse to five minutes. Submit.

RUBRIC — 100 POINTS

25

Accuracy of classification

Correct layer, defensible boundary, complete IPO-feedback-control

20

Depth of dynamics analysis

Evidence of real user contact, not assumption

15

Role reflection

Specific, self-aware, names a real exposure

25

Quality of the decision matrix

Weights justified, scores consistent, arithmetic correct

15

Defence & presentation

Five minutes, clear recommendation, handles one challenge

Late work follows the course policy. If you cannot access a real organisation, tell me by Day 2 and I will assign you a documented case instead.

Questions?

Bring one system you already work with. We will classify it together.

NEXT WEEK — WEEK 3

Understanding and Modeling Organizational Systems

Context-level data flow diagrams, entity-relationship models, use cases
— and organisational culture.

BEFORE YOU GO

- Download the Decision Brief worksheet from Classroom
- Choose your system by Day 1 — access is the constraint
- Post your chosen system in the Classroom thread